

B2B MULTI-TENANT SAAS CONTROL PLANE

Database Schema Documentation

96/100 Quality Score

Grade A Schema
Grade

40 Tables Generated

28 Rules Applied

Generated On	September 05, 2026 at 10:07
Domain	General
Scale	Enterprise
GST Compliance	Yes
AI Provider	Together — openai/gpt-oss-120b

1. PROJECT OVERVIEW

Provide a scalable, secure control plane for managing tenants, users, RBAC, subscriptions, billing, and integrations in a multi-tenant SaaS environment

Modules

1. Infrastructure	System registry, unique ID mapping, and configuration
`unique_id_header_all`	Centralised business ID registry for tracking every entity across all modules
`factory_reset_all`	Global platform feature flags and kill-switches singleton
2. Tenant Management	Core CRUD for tenant records, data store allocation, and per-tenant configuration.
`tenant_header_all`	Primary tenant organization record
`tenant_archive_all`	Historical archive mirror of tenant_header_all
`data_store_configuration_all`	Isolated data partition allocated to a tenant
`configuration_configuration_all`	Key-value settings namespace for a tenant (feature toggles, plan limits, etc.)
3. User & Identity	Handles user accounts, invitations, API keys, and tenant administrator role.
`user_header_all`	Individual user account belonging to a tenant
`user_archive_all`	Historical archive mirror of user_header_all
`user_invitation_transaction_all`	Invitation sent to a prospective user with role selection and activation token
`user_invitation_life_cycle_all`	State transition log and audit trail for user_invitation_transaction_all
`api_key_details_all`	Access token / API key issued to a user for programmatic access
`tenant_administrator_header_all`	User with administrative privileges for a tenant (specialized role)
`tenant_administrator_archive_all`	Historical archive mirror of tenant_administrator_header_all
4. RBAC Engine	Role definitions, assignments, and permission enforcement.
`role_header_all`	RBAC role definition (name, permissions, scope)
`role_archive_all`	Historical archive mirror of role_header_all
`role_assignment_all`	Link between a User and a Role within a tenant
5. Subscription & Billing	Subscription lifecycle, plan changes, usage aggregation, invoicing and ledger accounting.
`subscription_header_all`	Active subscription record linking a tenant to a plan and billing cycle
`subscription_archive_all`	Historical archive mirror of subscription_header_all

<code>`subscription_request_transaction_all`</code>	Request to create or change a subscription plan for a tenant
<code>`subscription_request_life_cycle_all`</code>	State transition log and audit trail for <code>subscription_request_transaction_all</code>
<code>`plan_change_request_transaction_all`</code>	Request to upgrade or downgrade a tenant's subscription plan
<code>`usage_record_transaction_all`</code>	Aggregated usage metrics for a tenant within a billing period
<code>`invoice_transaction_all`</code>	Generated billing invoice (PDF reference, amount, due date)
<code>`ledger_entry_transaction_all`</code>	Financial transaction line (debit/credit) generated by LedgerEngine
<code>`ledger_entry_life_cycle_all`</code>	State transition log and audit trail for <code>ledger_entry_transaction_all</code>
<code>`account_balance_details_all`</code>	Current monetary balance for a tenant account
<code>`credit_adjustment_request_transaction_all`</code>	Manual credit adjustment request submitted by finance
6. Approval Engine	Generic multi-stage approval workflow used by subscription changes, credit adjustments, etc.
<code>`approval_request_transaction_all`</code>	Record of a multi-stage approval workflow (status, approvers, decisions)
<code>`approval_request_life_cycle_all`</code>	State transition log and audit trail for <code>approval_request_transaction_all</code>
<code>`approver_all`</code>	User or role that can approve an ApprovalRequest
7. Audit & Compliance	Immutable audit logging, query interface, and compliance-related event tracking.
<code>`audit_log_transaction_all`</code>	Immutable audit trail entries for all system actions
<code>`audit_query_all`</code>	Parameters for retrieving audit log entries (tenant, time range, filters)
<code>`security_event_transaction_all`</code>	Detected security incident (e.g., suspicious login)
<code>`alert_transaction_all`</code>	Notification record generated for a security event, including recipients and status
<code>`alert_life_cycle_all`</code>	State transition log and audit trail for <code>alert_transaction_all</code>

Quality Validation

Score	96/100
Grade	A
Status	■ PASSED
Total Issues	2
Tables Found	40

2. TABLE-BY-TABLE DOCUMENTATION

For each table: purpose, column definitions, when to INSERT, when to UPDATE, and relationships.

1. `unique\_id\_header\_all`

Central registry for all business IDs. One row per entity type.

Column	Type	Notes
`id`	INT	System surrogate key
`table_name`	VARCHAR(100)	Table this ID sequence is for
`id_for`	VARCHAR(50)	Entity this ID represents
`prefix`	VARCHAR(20)	Business ID prefix (e.g. EMP, ORD, FLT)
`last_id`	VARCHAR(15)	Last issued sequence number
`business_id`	VARCHAR(20)	Human-readable business ID (e.g. UI00001)
`domain_code`	VARCHAR(30)	Domain specific code for categorisation
`effective_date`	DATETIME	Date from which this ID becomes active
`expiry_date`	DATETIME	Date after which this ID is no longer valid
`amount_limit`	DECIMAL(10,2)	Maximum monetary amount allowed for this entity
`sgst_amount`	DECIMAL(10,2)	State GST amount associated with the entity
`cgst_amount`	DECIMAL(10,2)	Central GST amount associated with the entity
`igst_amount`	DECIMAL(10,2)	Integrated GST amount associated with the entity
`sgst_percentage`	DECIMAL(5,2)	State GST rate applied
`cgst_percentage`	DECIMAL(5,2)	Central GST rate applied
`total_child_count`	INT	Aggregate count of child records linked to this entity
`last_child_id`	VARCHAR(20)	Business ID of the most recently created child record
`added_by`	INT	User ID who created this registry entry
`status`	INT	1=active, 2=inactive, 3=retired, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when

Once per entity type during initial setup. Example: register 'student_header_all' with prefix 'STU-'.

■ UPDATE when

Increment last_id and modified_on every time a new business ID is generated.

2. `factory\_reset\_all`

Supporting table: factory_reset_all

Column	Type	Notes
`id`	INT	System surrogate key
`app_version`	VARCHAR(20)	Current deployed application version
`maintenance_mode`	TINYINT	1=maintenance mode ON, 0=OFF
`api_throttle_enabled`	TINYINT	1=API throttling active, 0=inactive
`feature_x_enabled`	TINYINT	Feature X toggle (1=enabled, 0=disabled)
`feature_y_enabled`	TINYINT	Feature Y toggle (1=enabled, 0=disabled)
`max_concurrent_jobs`	INT	Maximum number of concurrent background jobs allowed
`default_currency`	VARCHAR(5)	Default currency code for financial transactions
`audit_retention_days`	INT	Number of days audit logs are retained
`password_policy_strength`	INT	Password strength level (1=low ... 5=high)
`data_encryption_enabled`	TINYINT	1=Data at rest encrypted, 0=not encrypted
`sgst_amount`	DECIMAL(10,2)	State GST amount used for platform■wide calculations
`cgst_amount`	DECIMAL(10,2)	Central GST amount used for platform■wide calculations
`igst_amount`	DECIMAL(10,2)	Integrated GST amount used for platform■wide calculations
`sgst_percentage`	DECIMAL(5,2)	Default State GST rate
`cgst_percentage`	DECIMAL(5,2)	Default Central GST rate
`closing_balance`	DECIMAL(12,2)	Platform■wide closing balance placeholder
`status`	INT	1=active, 2=inactive, 3=archived, 4=deleted
`added_by`	INT	User ID who created the singleton record
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when When the corresponding business event occurs.

■ UPDATE when When the record status or key fields change.

3. `tenant\_header\_all`

Master entity table for Tenant records. One row per tenant entity.

Column	Type	Notes
`id`	INT	System surrogate key

`tenant_id`	VARCHAR(20)	Human-readable business ID (e.g. TEN-00001)
`name`	VARCHAR(255)	Legal name of the tenant organization
`domain`	VARCHAR(255)	Primary internet domain used by the tenant
`contact_email`	VARCHAR(255)	Primary contact email address
`contact_phone`	VARCHAR(50)	Primary contact phone number
`address_line1`	VARCHAR(255)	First line of mailing address
`address_line2`	VARCHAR(255)	Second line of mailing address
`city`	VARCHAR(100)	City of the tenant address
`state`	VARCHAR(100)	State/Province of the tenant address
`postal_code`	VARCHAR(20)	Postal/ZIP code
`country`	VARCHAR(100)	Country of the tenant
`registration_date`	DATE	Date when tenant was legally registered
`plan_code`	VARCHAR(50)	Subscription plan code assigned to tenant
`subscription_start`	DATE	Start date of current subscription period
`subscription_end`	DATE	End date of current subscription period
`total_users`	INT	Aggregate count of user accounts belonging to tenant
`total_data_stores`	INT	Aggregate count of data stores allocated to tenant
`last_data_store_id`	VARCHAR(20)	Business ID of the most recently created data store for tenant
`added_by`	INT	User ID who created this tenant record
`status`	INT	1=active, 2=inactive, 3=suspended, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when

When a new tenant is registered or onboarded into the system.

■ UPDATE when

When tenant profile information changes. Always copy old row to tenant_archive_all first.

4. `tenant\_archive\_all`

Historical mirror of tenant_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key (archived copy)
`tenant_id`	VARCHAR(20)	Business ID of tenant
`name`	VARCHAR(255)	Legal name of the tenant organization

`domain`	VARCHAR(255)	Primary internet domain used by the tenant
`contact_email`	VARCHAR(255)	Primary contact email address
`contact_phone`	VARCHAR(50)	Primary contact phone number
`address_line1`	VARCHAR(255)	First line of mailing address
`address_line2`	VARCHAR(255)	Second line of mailing address
`city`	VARCHAR(100)	City of the tenant address
`state`	VARCHAR(100)	State/Province of the tenant address
`postal_code`	VARCHAR(20)	Postal/ZIP code
`country`	VARCHAR(100)	Country of the tenant
`registration_date`	DATE	Date when tenant was legally registered
`plan_code`	VARCHAR(50)	Subscription plan code assigned to tenant
`subscription_start`	DATE	Start date of current subscription period
`subscription_end`	DATE	End date of current subscription period
`total_users`	INT	Aggregate count of user accounts belonging to tenant
`total_data_stores`	INT	Aggregate count of data stores allocated to tenant
`last_data_store_id`	VARCHAR(20)	Business ID of the most recently created data store for tenant
`added_by`	INT	User ID who created the original tenant record
`status`	INT	Status at time of archiving (1=active, 2=inactive, 3=suspended, 4=deleted)
`created_on`	DATETIME	Original record creation timestamp
`modified_on`	DATETIME	Original last modification timestamp
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version
■ INSERT when	Every time a row in tenant_header_all is about to be UPDATED. Copy the old row here first.	
■ UPDATE when	Never update archive rows — they are immutable historical records.	

5. `data\_store\_configuration\_all`

Temporal configuration for data_store. Stores settings with effective date ranges.

Column	Type	Notes
`id`	INT	System surrogate key
`tenant_id`	INT	FK to tenant_header_all.id

<code>`data_store_id`</code>	VARCHAR(20)	Business ID of the data store (generated from unique_id_header_all)
<code>`entity_type`</code>	VARCHAR(50)	Type of entity the configuration applies to (e.g., DOCUMENTS, IMAGES)
<code>`storage_quota_gb`</code>	DECIMAL(10,2)	Maximum storage quota allocated (GB)
<code>`retention_days`</code>	INT	Number of days data is retained before automatic purge
<code>`is_encrypted`</code>	TINYINT(1)	1=encrypted at rest, 0=not encrypted
<code>`effective_from`</code>	DATE	Date from which this configuration becomes effective
<code>`effective_to`</code>	DATE	Date after which this configuration expires (NULL = indefinite)
<code>`updated_by`</code>	INT	User ID who last updated this configuration
<code>`approved_by`</code>	INT	User ID who approved the configuration change
<code>`status`</code>	INT	1=active, 2=inactive, 3=revoked
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Last modification timestamp

■ INSERT when	When a new configuration is set. Insert new row with new from_date. Do not update the old row — let it expire.
■ UPDATE when	Update status to 0 (past) when a newer configuration takes over.

6. ``configuration_configuration_all``

Temporal configuration for configuration. Stores settings with effective date ranges.

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`tenant_id`</code>	INT	FK to tenant_header_all.id
<code>`feature_name`</code>	VARCHAR(100)	Name of the feature toggle or setting
<code>`is_enabled`</code>	TINYINT(1)	1=feature enabled, 0=disabled
<code>`limit_value`</code>	INT	Numeric limit associated with the feature (e.g., max API calls)
<code>`effective_from`</code>	DATE	Date from which this configuration becomes effective
<code>`effective_to`</code>	DATE	Date after which this configuration expires (NULL = indefinite)
<code>`updated_by`</code>	INT	User ID who last updated this configuration
<code>`approved_by`</code>	INT	User ID who approved the configuration change
<code>`status`</code>	INT	1=active, 2=inactive, 3=deprecated

<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Last modification timestamp

■ INSERT when	When a new configuration is set. Insert new row with new <code>from_date</code> . Do not update the old row — let it expire.
■ UPDATE when	Update status to 0 (past) when a newer configuration takes over.

7. ``user_header_all``

Master entity table for User records. One row per user entity.

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status`</code>	INT	1=active, 2=inactive, 3=locked, 4=deleted
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Last modification timestamp
<code>`user_id`</code>	VARCHAR(20)	Human-readable business ID (e.g. U-00001)
<code>`tenant_id`</code>	INT	FK to <code>tenant_header_all</code>
<code>`added_by`</code>	INT	User ID who created this record
<code>`first_name`</code>	VARCHAR(100)	User first name
<code>`last_name`</code>	VARCHAR(100)	User last name
<code>`email`</code>	VARCHAR(255)	User email address (login)
<code>`phone`</code>	VARCHAR(20)	Contact phone number
<code>`role`</code>	VARCHAR(50)	Application role (e.g. admin, manager, user)
<code>`is_admin`</code>	TINYINT	Flag indicating tenant administrator (1=yes,0=no)
<code>`password_hash`</code>	VARCHAR(255)	Hashed password
<code>`password_last_changed_on`</code>	DATETIME	When password was last changed
<code>`last_login_on`</code>	DATETIME	Timestamp of last successful login
<code>`failed_login_count`</code>	INT	Consecutive failed login attempts
<code>`total_login_count`</code>	INT	Cumulative successful logins
<code>`wallet_balance`</code>	DECIMAL(12,2)	Current wallet balance for the user
<code>`total_invitations_sent`</code>	INT	Number of invitations this user has sent
<code>`total_api_keys`</code>	INT	Number of API keys owned by the user
<code>`last_invitation_sent_on`</code>	DATETIME	When the last invitation was sent
<code>`address_line1`</code>	VARCHAR(255)	Primary address line
<code>`city`</code>	VARCHAR(100)	City
<code>`state`</code>	VARCHAR(100)	State / Province

<code>`country`</code>	VARCHAR(100)	Country
<code>`zip_code`</code>	VARCHAR(20)	Postal code

■ INSERT when	When a new user is registered or onboarded into the system.
■ UPDATE when	When user profile information changes. Always copy old row to <code>user_archive_all</code> first.

8. ``user_archive_all``

Historical mirror of `user_header_all`. Stores a full copy of every row before it was changed.

Column	Type	Notes
<code>`id`</code>	INT	Surrogate key from source (not PK in archive)
<code>`status`</code>	INT	Status at time of archive
<code>`created_on`</code>	DATETIME	Original creation timestamp
<code>`modified_on`</code>	DATETIME	Original modification timestamp
<code>`user_id`</code>	VARCHAR(20)	Business ID of the user
<code>`tenant_id`</code>	INT	Tenant reference
<code>`added_by`</code>	INT	Creator user ID
<code>`first_name`</code>	VARCHAR(100)	User first name
<code>`last_name`</code>	VARCHAR(100)	User last name
<code>`email`</code>	VARCHAR(255)	User email address
<code>`phone`</code>	VARCHAR(20)	Contact phone number
<code>`role`</code>	VARCHAR(50)	Application role
<code>`is_admin`</code>	TINYINT	Tenant administrator flag
<code>`password_hash`</code>	VARCHAR(255)	Hashed password
<code>`password_last_changed_on`</code>	DATETIME	Password last changed timestamp
<code>`last_login_on`</code>	DATETIME	Last successful login timestamp
<code>`failed_login_count`</code>	INT	Failed login attempts count
<code>`total_login_count`</code>	INT	Total successful logins
<code>`wallet_balance`</code>	DECIMAL(12,2)	Wallet balance at archive time
<code>`total_invitations_sent`</code>	INT	Invitations sent count at archive time
<code>`total_api_keys`</code>	INT	API keys count at archive time
<code>`last_invitation_sent_on`</code>	DATETIME	Timestamp of last invitation sent
<code>`address_line1`</code>	VARCHAR(255)	Primary address line
<code>`city`</code>	VARCHAR(100)	City
<code>`state`</code>	VARCHAR(100)	State / Province
<code>`country`</code>	VARCHAR(100)	Country

<code>`zip_code`</code>	VARCHAR(20)	Postal code
<code>`archived_on`</code>	DATETIME	When this version was archived
<code>`archived_by`</code>	INT	User who performed the archive
<code>`archive_reason`</code>	VARCHAR(255)	Reason for archiving this version
■ INSERT when	Every time a row in <code>user_header_all</code> is about to be UPDATED. Copy the old row here first.	
■ UPDATE when	Never update archive rows — they are immutable historical records.	

9. ``user_invitation_transaction_all``

Event/ledger table for `user_invitation` operations. Append-only record of all events.

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status`</code>	INT	1=sent, 2=accepted, 3=expired, 4=revoked, 5=failed
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Last modification timestamp
<code>`month_year`</code>	VARCHAR(7)	YYYY-MM for partitioning
<code>`transaction_ref`</code>	VARCHAR(30)	Human-readable transaction ID
<code>`invitation_id`</code>	VARCHAR(20)	Business ID for this invitation
<code>`inviter_user_id`</code>	INT	User who sent the invitation
<code>`invitee_email`</code>	VARCHAR(255)	Email of the prospective user
<code>`role_assigned`</code>	VARCHAR(50)	Role to be granted upon acceptance
<code>`token`</code>	VARCHAR(100)	Activation token sent to invitee
<code>`token_expires_on`</code>	DATETIME	When the activation token expires
<code>`invitation_sent_on`</code>	DATETIME	Timestamp when invitation was sent
<code>`invitation_accepted_on`</code>	DATETIME	Timestamp when invitation was accepted
<code>`approval_1_by`</code>	INT	User ID who performed first approval (if any)
<code>`approval_1_on`</code>	DATETIME	Timestamp of first approval
<code>`approval_1_status`</code>	INT	Status after first approval (1=approved, 2=rejected)
<code>`closing_balance`</code>	DECIMAL(12, 2)	Running balance after this transaction
<code>`added_by`</code>	INT	User who recorded this transaction
<code>`device_id`</code>	VARCHAR(50)	Device identifier from which action originated
<code>`ip_address`</code>	VARCHAR(45)	IP address of the requester

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

10. `user\_invitation\_life\_cycle\_all`

Status change audit trail for user_invitation_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`invitation_id`	VARCHAR(20)	Business ID of the invitation
`previous_status`	INT	1=sent, 2=accepted, 3=expired, 4=revoked, 5=failed
`new_status`	INT	1=sent, 2=accepted, 3=expired, 4=revoked, 5=failed
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the change
`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks

■ INSERT when	Every time the status column in user_invitation_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

11. `api\_key\_details\_all`

Supporting table: api_key_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=revoked, 3=expired, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`user_id`	INT	FK to user_header_all.id — owner of the API key

`tenant_id`	INT	FK to tenant_header_all.id – tenant context for the key
`sort_order`	INT	Line number / sequence for ordering multiple keys per user
`api_key`	VARCHAR(64)	Public API key identifier
`secret_hash`	VARCHAR(255)	Hashed secret used for HMAC verification
`expires_on`	DATETIME	Expiration timestamp of the API key
`is_active`	TINYINT	1=currently usable, 0=inactive
`usage_counter`	BIGINT	Number of successful calls made with this key
`last_used_on`	DATETIME	Timestamp of the most recent successful use
`created_by`	INT	User ID who created this API key (FK to user_header_all.id)
`month_year`	VARCHAR(7)	Pre-tagged YYYYMM for analytics partitioning

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

12. `tenant\_administrator\_header\_all`

Master entity table for Tenant Administrator records. One row per tenant administrator entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=revoked, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`ta_id`	VARCHAR(20)	Human-readable business ID for tenant administrator (e.g. TA00001)
`tenant_id`	INT	FK to tenant_header_all.id – tenant to which the admin belongs
`user_id`	INT	FK to user_header_all.id – user granted admin role
`role_code`	VARCHAR(30)	Code representing admin role level (e.g. SUPER, MANAGER, AUDITOR)
`assigned_on`	DATETIME	Timestamp when admin rights were granted
`revoked_on`	DATETIME	Timestamp when admin rights were revoked, if any
`is_primary`	TINYINT	1=primary admin for tenant, 0=secondary

`total_api_keys`	INT	Aggregate counter of API keys issued to this admin
`last_api_key_on`	DATETIME	Timestamp of the most recent API key issuance
`total_sessions`	INT	Aggregate counter of login sessions for this admin
`last_login_on`	DATETIME	Timestamp of the most recent successful login
`notes`	VARCHAR(500)	Free-form notes about the admin assignment
`added_by`	INT	User ID who created this admin record (FK to user_header_all.id)

■ INSERT when	When a new tenant administrator is registered or onboarded into the system.
■ UPDATE when	When tenant administrator profile information changes. Always copy old row to tenant_administrator_archive_all first.

13. `tenant\_administrator\_archive\_all`

Historical mirror of tenant_administrator_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key (mirrored from header)
`status`	INT	Status at time of archiving
`created_on`	DATETIME	Original creation timestamp
`modified_on`	DATETIME	Original last modification timestamp
`ta_id`	VARCHAR(20)	Business ID of the tenant administrator
`tenant_id`	INT	Tenant reference (mirrored)
`user_id`	INT	User reference (mirrored)
`role_code`	VARCHAR(30)	Role code at time of archiving
`assigned_on`	DATETIME	When admin rights were originally granted
`revoked_on`	DATETIME	When admin rights were revoked, if any
`is_primary`	TINYINT	Primary admin flag at time of archiving
`total_api_keys`	INT	API key count at time of archiving
`last_api_key_on`	DATETIME	Most recent API key issuance timestamp
`total_sessions`	INT	Session count at time of archiving
`last_login_on`	DATETIME	Most recent login timestamp
`notes`	VARCHAR(500)	Archived notes
`added_by`	INT	User who originally added the record
`archived_on`	DATETIME	When this version was archived

`archived_by`	INT	User who performed the archival
`archive_reason`	VARCHAR(255)	Reason for archiving this version
■ INSERT when	Every time a row in tenant_administrator_header_all is about to be UPDATED. Copy the old row here first.	
■ UPDATE when	Never update archive rows — they are immutable historical records.	

14. `role\_header\_all`

Master entity table for Role records. One row per role entity.

Column	Type	Notes
`id`	INT	System surrogate key
`role_id`	VARCHAR(20)	Human-readable business ID (e.g. ROL00001)
`role_name`	VARCHAR(50)	Descriptive name of the role
`role_description`	VARCHAR(255)	Full description of role responsibilities
`role_scope`	VARCHAR(30)	Scope of role applicability (e.g. GLOBAL, TENANT, DEPARTMENT)
`permission_codes`	VARCHAR(500)	Comma-separated list of permission codes assigned to this role
`is_default`	TINYINT	1=default role for new users, 0=non-default
`max_user_assignments`	INT	Maximum number of users that can be assigned this role (0 = unlimited)
`effective_from`	DATETIME	Date-time when role becomes active
`effective_to`	DATETIME	Date-time when role expires (NULL = indefinite)
`total_user_assignments`	INT	Aggregate counter of total user assignments to this role
`last_assigned_on`	DATETIME	Timestamp of most recent user assignment
`added_by`	INT	User ID who created this role record
`tenant_id`	INT	Tenant to which this role belongs
`status`	INT	1=active, 2=inactive, 3=deleted, 4=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
■ INSERT when	When a new role is registered or onboarded into the system.	
■ UPDATE when	When role profile information changes. Always copy old row to role_archive_all first.	

15. `role\_archive\_all`

Historical mirror of role_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	Surrogate key (mirrored from source)
`role_id`	VARCHAR(20)	Business ID of the role
`role_name`	VARCHAR(50)	Descriptive name of the role
`role_description`	VARCHAR(255)	Full description of role responsibilities
`role_scope`	VARCHAR(30)	Scope of role applicability
`permission_codes`	VARCHAR(500)	Comma-separated list of permission codes
`is_default`	TINYINT	1=default role, 0=non-default
`max_user_assignments`	INT	Maximum allowed user assignments
`effective_from`	DATETIME	Activation timestamp
`effective_to`	DATETIME	Expiration timestamp
`total_user_assignments`	INT	Aggregate counter of assignments
`last_assigned_on`	DATETIME	Most recent assignment timestamp
`added_by`	INT	Creator user ID
`tenant_id`	INT	Owning tenant ID
`status`	INT	Status at time of archiving
`created_on`	DATETIME	Original creation timestamp
`modified_on`	DATETIME	Original modification timestamp
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version
■ INSERT when	Every time a row in role_header_all is about to be UPDATED. Copy the old row here first.	
■ UPDATE when	Never update archive rows — they are immutable historical records.	

16. `role\_assignment\_all`

Supporting table: role_assignment_all

Column	Type	Notes
`id`	INT	System surrogate key
`assignment_id`	VARCHAR(20)	Human-readable business ID for the assignment (e.g. ASS00001)
`user_id`	INT	User to which the role is assigned
`role_id`	INT	Role being assigned
`tenant_id`	INT	Tenant context of the assignment

`assigned_on`	DATETIME	Timestamp when assignment became effective
`assigned_by`	INT	User who performed the assignment
`expires_on`	DATETIME	Optional expiration timestamp for the assignment
`assignment_status`	INT	1=active, 2=revoked, 3=expired, 4=pending
`notes`	VARCHAR(255)	Free-form notes about the assignment
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

17. `subscription\_header\_all`

Master entity table for Subscription records. One row per subscription entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=suspended, 3=cancelled, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`subscription_id`	VARCHAR(20)	Human-readable business ID (e.g. SUB-00001)
`tenant_id`	INT	FK to tenant_header_all.id
`plan_id`	INT	FK to plan_header_all (plan definition)
`billing_cycle`	VARCHAR(20)	e.g. monthly, yearly
`start_date`	DATE	Subscription start date
`end_date`	DATE	Subscription end date (null if ongoing)
`next_renewal_date`	DATE	Next scheduled renewal date
`currency_code`	CHAR(3)	ISO currency code
`base_amount`	DECIMAL(10,2)	Base subscription amount before taxes
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied

`igst_percentage`	DECIMAL(5,2)	IGST rate applied
`total_usage`	BIGINT	Aggregated usage units for the subscription
`total_invoices`	INT	Count of invoices generated for this subscription
`last_invoice_on`	DATETIME	Timestamp of the most recent invoice
`added_by`	INT	User ID who created this subscription

■ INSERT when	When a new subscription is registered or onboarded into the system.
■ UPDATE when	When subscription profile information changes. Always copy old row to subscription_archive_all first.

18. `subscription\_archive\_all`

Historical mirror of subscription_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	Surrogate key from source (not PK here)
`status`	INT	Status at time of archive
`created_on`	DATETIME	Original creation timestamp
`modified_on`	DATETIME	Original last modification timestamp
`subscription_id`	VARCHAR(20)	Business ID
`tenant_id`	INT	Tenant reference
`plan_id`	INT	Plan reference
`billing_cycle`	VARCHAR(20)	Billing cycle type
`start_date`	DATE	Subscription start date
`end_date`	DATE	Subscription end date
`next_renewal_date`	DATE	Next renewal date
`currency_code`	CHAR(3)	Currency code
`base_amount`	DECIMAL(10,2)	Base amount
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate
`cgst_percentage`	DECIMAL(5,2)	CGST rate
`igst_percentage`	DECIMAL(5,2)	IGST rate
`total_usage`	BIGINT	Aggregated usage
`total_invoices`	INT	Invoice count
`last_invoice_on`	DATETIME	Most recent invoice timestamp

`added_by`	INT	Creator user ID
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving

■ INSERT when	Every time a row in subscription_header_all is about to be UPDATED. Copy the old row here first.
■ UPDATE when	Never update archive rows — they are immutable historical records.

19. `subscription\_request\_transaction\_all`

Event/ledger table for subscription_request operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=pending, 2=approved, 3=rejected, 4=processed
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`subscription_id`	VARCHAR(20)	Target subscription business ID (if update)
`tenant_id`	INT	Tenant requesting the change
`requested_by`	INT	User ID who initiated the request
`device_id`	VARCHAR(50)	Device identifier of the requester
`ip_address`	VARCHAR(45)	IP address of the requester
`request_type`	VARCHAR(20)	CREATE, UPDATE, CANCEL
`before_plan_id`	INT	Plan ID before change
`after_plan_id`	INT	Plan ID after change
`before_amount`	DECIMAL(10,2)	Amount before change
`after_amount`	DECIMAL(10,2)	Amount after change
`sgst_amount`	DECIMAL(10,2)	State GST amount for this request
`cgst_amount`	DECIMAL(10,2)	Central GST amount for this request
`igst_amount`	DECIMAL(10,2)	Integrated GST amount for this request
`approval_1_by`	INT	User ID of first approver
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected, 3=escalated
`approval_2_by`	INT	User ID of second approver

`approval_2_on`	DATETIME	Timestamp of second approval
`approval_2_status`	INT	1=approved, 2=rejected, 3=escalated
`closing_balance`	DECIMAL(12, 2)	Running balance after this transaction

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

20. `subscription\_request\_life\_cycle\_all`

Status change audit trail for subscription_request_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`subscription_request_id`	INT	FK to subscription_request_transaction_all.id
`previous_status`	INT	Status before this change (1=pending,2=approved,3=rejected,4=processed)
`new_status`	INT	Status after this change (same codes as previous_status)
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the transition
`changed_on`	DATETIME	When the transition occurred
`trigger_event`	VARCHAR(100)	What triggered this transition (e.g., approval, timeout)
`remarks`	TEXT	Additional remarks

■ INSERT when	Every time the status column in subscription_request_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

21. `plan\_change\_request\_transaction\_all`

Event/ledger table for plan_change_request operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key

`status`	INT	1=pending, 2=approved, 3=rejected, 4=cancelled
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`tenant_id`	INT	FK to tenant_header_all.id
`subscription_id`	VARCHAR(20)	Business ID of the subscription being changed
`requestor_user_id`	INT	User who initiated the plan change
`previous_plan_id`	VARCHAR(20)	Business ID of the current plan before change
`new_plan_id`	VARCHAR(20)	Business ID of the target plan after change
`previous_amount`	DECIMAL(10,2)	Amount of current plan
`new_amount`	DECIMAL(10,2)	Amount of target plan
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`approval_1_by`	INT	User ID of first approver
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected
`added_by`	INT	User ID who inserted this record
`device_id`	VARCHAR(50)	Device identifier used for the request
`ip_address`	VARCHAR(45)	IP address of the requester

■ INSERT when

Every time a new transaction, payment, or event occurs. Each row represents one atomic event.

■ UPDATE when

Update status column only (e.g. pending → paid). Never modify financial columns after insert.

22. `usage\_record\_transaction\_all`

Event/ledger table for usage_record operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=recorded, 2=adjusted, 3=invalidated
`created_on`	DATETIME	Record creation timestamp

`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`tenant_id`	INT	FK to tenant_header_all
`subscription_id`	VARCHAR(20)	Business ID of the subscription
`usage_metric`	VARCHAR(50)	Name of the usage metric (e.g., API_CALLS)
`usage_quantity`	DECIMAL(12,2)	Aggregated quantity for the period
`unit_price`	DECIMAL(10,2)	Price per unit of the metric
`total_amount`	DECIMAL(10,2)	Total amount = quantity * unit_price
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`added_by`	INT	User ID who recorded the usage
`device_id`	VARCHAR(50)	Device identifier used for recording
`ip_address`	VARCHAR(45)	IP address of the source system
■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.	
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.	

23. `invoice\_transaction\_all`

Event/ledger table for invoice operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=issued, 2=paid, 3=overdue, 4=cancelled
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable invoice ID
`tenant_id`	INT	FK to tenant_header_all.id
`subscription_id`	VARCHAR(20)	Business ID of the subscription billed
`invoice_number`	VARCHAR(30)	Official invoice number
`invoice_date`	DATETIME	Date the invoice was generated

`due_date`	DATETIME	Payment due date
`subtotal_amount`	DECIMAL(10,2)	Sum of line items before tax
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`total_amount`	DECIMAL(10,2)	Grand total payable
`pdf_url`	VARCHAR(255)	Link to stored PDF version of the invoice
`approval_1_by`	INT	User ID of first approver (if manual approval required)
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected
`added_by`	INT	User ID who created the invoice record
`device_id`	VARCHAR(50)	Device identifier used for creation
`ip_address`	VARCHAR(45)	IP address of the creator

■ INSERT when

Every time a new transaction, payment, or event occurs. Each row represents one atomic event.

■ UPDATE when

Update status column only (e.g. pending → paid). Never modify financial columns after insert.

24. `ledger\_entry\_transaction\_all`

Event/ledger table for ledger_entry operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=posted, 2=reversed, 3=failed
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable ledger entry ID
`tenant_id`	INT	FK to tenant_header_all.id
`invoice_id`	VARCHAR(30)	Business ID of related invoice, if any
`entry_type`	VARCHAR(20)	Debit or Credit
`amount`	DECIMAL(10,2)	Transaction amount
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount

`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`opening_balance`	DECIMAL(12,2)	Balance before this entry
`closing_balance`	DECIMAL(12,2)	Balance after this entry
`batch_id`	VARCHAR(30)	Reconciliation batch identifier
`approval_1_by`	INT	User ID of first approver
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected
`added_by`	INT	User ID who posted the ledger entry
`device_id`	VARCHAR(50)	Device identifier used for posting
`ip_address`	VARCHAR(45)	IP address of the posting system

■ INSERT when

Every time a new transaction, payment, or event occurs. Each row represents one atomic event.

■ UPDATE when

Update status column only (e.g. pending → paid). Never modify financial columns after insert.

25. `ledger\_entry\_life\_cycle\_all`

Status change audit trail for ledger_entry_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`ledger_entry_id`	INT	FK to ledger_entry_transaction_all.id
`previous_status`	INT	Previous state of the ledger entry (1=created,2=posted,3=reversed,4=failed)
`new_status`	INT	New state of the ledger entry (1=created,2=posted,3=reversed,4=failed)
`reason`	VARCHAR(500)	Reason for state change
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of the state change
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks
`status`	INT	1=active,2=inactive,3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when

Every time the status column in ledger_entry_header_all changes. Record previous_status, new_status, and datetime.

■ UPDATE when

Never update life cycle rows — append only.

26. `account\_balance\_details\_all`

Supporting table: account_balance_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`tenant_id`	INT	FK to tenant_header_all.id
`sort_order`	INT	Sequence number for line item ordering
`balance_type`	VARCHAR(30)	Type of balance (e.g., prepaid, credit, escrow)
`amount`	DECIMAL(12,2)	Monetary amount for this line item
`description`	VARCHAR(255)	Human-readable description of the balance line
`effective_date`	DATE	Date from which this balance line is effective
`status`	INT	1=active,2=inactive,3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when

When the corresponding business event occurs.

■ UPDATE when

When the record status or key fields change.

27. `credit\_adjustment\_request\_transaction\_all`

Event/ledger table for credit_adjustment_request operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`month_year`	VARCHAR(7)	YYYY-MM for partitioning and analytics
`request_id`	VARCHAR(20)	Human-readable request ID (e.g., CADJ-00001)
`tenant_id`	INT	FK to tenant_header_all.id
`requested_by_user_id`	INT	FK to user_header_all.id
`approved_by_user_id`	INT	FK to user_header_all.id
`before_balance`	DECIMAL(12,2)	Tenant balance before adjustment
`adjustment_amount`	DECIMAL(10,2)	Amount to be added (positive) or deducted (negative)
`after_balance`	DECIMAL(12,2)	Tenant balance after adjustment
`adjustment_type`	VARCHAR(30)	Type of adjustment (e.g., credit_note, manual_refund)

<code>`approval_status`</code>	INT	0=pending,1=approved,2=rejected
<code>`approval_1_by`</code>	INT	User ID of first approver
<code>`approval_1_on`</code>	DATETIME	Timestamp of first approval
<code>`approval_1_status`</code>	INT	Status of first approval (1=approved,2=rejected)
<code>`added_by`</code>	INT	User ID who created the record
<code>`device_id`</code>	VARCHAR(50)	Device identifier from which request originated
<code>`ip_address`</code>	VARCHAR(45)	IP address of the requester
<code>`status`</code>	INT	1=active,2=inactive,3=deleted
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Last modification timestamp

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

28. ``approver_all``

Supporting table: `approver_all`

Column	Type	Notes
<code>`id`</code>	INT	System surrogate key
<code>`status`</code>	INT	1=active, 2=inactive, 3=deleted, 4=suspended
<code>`created_on`</code>	DATETIME	Record creation timestamp
<code>`modified_on`</code>	DATETIME	Last modification timestamp
<code>`approver_id`</code>	VARCHAR(20)	Human-readable business ID (e.g. APR-00001)
<code>`user_id`</code>	INT	FK to <code>user_header_all.id</code> who is the approver
<code>`role_id`</code>	INT	FK to <code>role_header_all.id</code> defining approver role
<code>`department`</code>	VARCHAR(100)	Department to which the approver belongs
<code>`approval_limit`</code>	DECIMAL(12,2)	Maximum monetary amount the approver can authorize
<code>`approval_level`</code>	INT	Sequential level in multi-stage approval chain (1=first, 2=second, ...)
<code>`notification_email`</code>	VARCHAR(255)	Email address for approval notifications
<code>`phone_number`</code>	VARCHAR(20)	Contact phone number
<code>`is_default_approver`</code>	TINYINT(1)	1=default approver for its level, 0=otherwise

`effective_from`	DATETIME	Date from which approver is active
`effective_to`	DATETIME	Date after which approver is inactive (null = indefinite)
`added_by`	INT	User ID who created this approver record
`device_id`	VARCHAR(50)	Device identifier used at creation
`ip_address`	VARCHAR(45)	IP address of creator
■ INSERT when	When the corresponding business event occurs.	
■ UPDATE when	When the record status or key fields change.	

29. `approval\_request\_transaction\_all`

Event/ledger table for approval_request operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=pending, 2=partially_approved, 3=approved, 4=rejected, 5=cancelled
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`approval_request_id`	VARCHAR(20)	Human-readable business ID (e.g. ARQ-00001)
`month_year`	VARCHAR(7)	YYYYMM for partitioning
`transaction_ref`	VARCHAR(30)	External reference number for the request
`requester_id`	INT	FK to user_header_all.id who initiated the request
`target_entity`	VARCHAR(50)	Entity type being acted upon (e.g. subscription, credit_adjustment)
`target_entity_id`	VARCHAR(20)	Business ID of the target entity
`amount_before`	DECIMAL(10,2)	Value before change
`amount_after`	DECIMAL(10,2)	Value after change
`currency_code`	VARCHAR(3)	ISO currency code
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`closing_balance`	DECIMAL(12,2)	Running balance after this transaction
`approval_1_by`	INT	FK to approver_all.id who performed first approval

`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected, 3=escalated
`approval_2_by`	INT	FK to approver_all.id who performed second approval
`approval_2_on`	DATETIME	Timestamp of second approval
`approval_2_status`	INT	1=approved, 2=rejected, 3=escalated
`approval_3_by`	INT	FK to approver_all.id who performed third approval
`approval_3_on`	DATETIME	Timestamp of third approval
`approval_3_status`	INT	1=approved, 2=rejected, 3=escalated
`added_by`	INT	User ID who created this request
`device_id`	VARCHAR(50)	Device identifier used at creation
`ip_address`	VARCHAR(45)	IP address of creator
■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.	
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.	

30. `approval\_request\_life\_cycle\_all`

Status change audit trail for approval_request_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`approval_request_id`	INT	FK to approval_request_transaction_all.id
`previous_status`	INT	1=pending, 2=partially_approved, 3=approved, 4=rejected, 5=cancelled
`new_status`	INT	1=pending, 2=partially_approved, 3=approved, 4=rejected, 5=cancelled
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of status change
`trigger_event`	VARCHAR(100)	What triggered this transition (e.g., approval_1, timeout)
`remarks`	TEXT	Additional remarks

■ INSERT when	Every time the status column in approval_request_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

31. `audit\_log\_transaction\_all`

Event/ledger table for audit_log operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted [document all values]
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`table_name`	VARCHAR(100)	Name of the table being audited
`row_id`	VARCHAR(36)	Primary key of the audited row (UUID if applicable)
`action`	VARCHAR(10)	INSERT UPDATE DELETE
`before_state`	JSON	JSON snapshot of row before change
`after_state`	JSON	JSON snapshot of row after change
`actor_user_id`	INT	User ID who performed the action
`device_id`	VARCHAR(50)	Device identifier used for the action
`ip_address`	VARCHAR(45)	IP address of the actor
`user_agent`	VARCHAR(255)	User-agent string of the client
`added_by`	INT	User ID who created this audit record

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

32. `audit\_query\_all`

Supporting table: audit_query_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted [document all values]

`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`tenant_id`	INT	Tenant for which audit query is scoped
`query_name`	VARCHAR(100)	Descriptive name of the saved query
`month_year_start`	VARCHAR(7)	Start month (YYYY-MM) for query range
`month_year_end`	VARCHAR(7)	End month (YYYY-MM) for query range
`table_filter`	VARCHAR(200)	Comma-separated list of tables to include
`action_filter`	VARCHAR(100)	Comma-separated list of actions (INSERT,UPDATE,DELETE)
`user_id_filter`	INT	Filter by actor user ID (optional)
`ip_address_filter`	VARCHAR(45)	Filter by IP address (optional)
`additional_criteria`	JSON	Arbitrary extra criteria in JSON
`added_by`	INT	User ID who saved this query
`device_id`	VARCHAR(50)	Device identifier used to create the query
`ip_address`	VARCHAR(45)	IP address of the creator

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

33. `security\_event\_transaction\_all`

Event/ledger table for security_event operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted [document all values]
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`event_type`	VARCHAR(50)	Type of security event (e.g., suspicious_login, brute_force)
`severity`	INT	Severity level (1=low, 5=critical) [document all values]
`source_ip`	VARCHAR(45)	IP address where event originated
`destination_ip`	VARCHAR(45)	Target IP address (if applicable)
`user_id`	INT	User ID involved in the event
`device_id`	VARCHAR(50)	Device identifier involved

`user_agent`	VARCHAR(255)	User agent string of the client
`event_timestamp`	DATETIME	Exact time the event was detected
`description`	TEXT	Detailed description of the security incident
`remediation_status`	VARCHAR(30)	Current remediation status (e.g., pending, resolved)
`remediation_by`	INT	User ID who performed remediation
`remediation_on`	DATETIME	Timestamp when remediation was applied
`added_by`	INT	User ID who logged the security event
`device_id_logger`	VARCHAR(50)	Device that logged the event
`ip_address_logger`	VARCHAR(45)	IP address of the logging device

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

34. `alert\_transaction\_all`

Event/ledger table for alert operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted [document all values]
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human-readable transaction ID
`alert_type`	VARCHAR(50)	Type of alert (e.g., security_breach, compliance_violation)
`severity`	INT	Severity level (1=low, 5=critical) [document all values]
`related_event_id`	INT	FK to security_event_transaction_all.id
`recipient_user_id`	INT	User ID of alert recipient
`channel`	VARCHAR(30)	Delivery channel (email, sms, push)
`delivery_status`	VARCHAR(30)	Current delivery status (queued, sent, failed)
`delivery_attempts`	INT	Number of delivery attempts made
`last_attempt_on`	DATETIME	Timestamp of the last delivery attempt
`message_subject`	VARCHAR(200)	Subject line of the alert message
`message_body`	TEXT	Full alert message content

`added_by`	INT	User ID who generated the alert
`device_id`	VARCHAR(50)	Device identifier that generated the alert
`ip_address`	VARCHAR(45)	IP address of the device generating the alert

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

35. `alert\_life\_cycle\_all`

Status change audit trail for alert_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`alert_id`	INT	FK to alert_transaction_all.id
`previous_status`	INT	Previous status code (e.g., 1=active, 2=resolved, 3=escalated, 4=closed)
`new_status`	INT	New status code after transition (e.g., 1=active, 2=resolved, 3=escalated, 4=closed)
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp when the status change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted [document all values]
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when	Every time the status column in alert_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

36. `tenant\_administrator\_life\_cycle\_all`

Status change audit trail for tenant_administrator_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key

`tenant_id`	INT	FK to tenant_header_all.id
`user_id`	INT	FK to user_header_all.id
`previous_is_admin`	TINYINT	Previous admin flag (0/1)
`new_is_admin`	TINYINT	New admin flag (0/1)
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of the change
`reason`	VARCHAR(500)	Reason for change
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when

Every time the status column in tenant_administrator_header_all changes. Record previous_status, new_status, and datetime.

■ UPDATE when

Never update life cycle rows — append only.

37. `tenant\_life\_cycle\_all`

Status change audit trail for tenant_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`tenant_id`	INT	FK to tenant_header_all.id
`previous_status`	INT	Previous tenant status
`new_status`	INT	New tenant status
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of the change
`reason`	VARCHAR(500)	Reason for status change
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when

Every time the status column in tenant_header_all changes. Record previous_status, new_status, and datetime.

■ UPDATE when

Never update life cycle rows — append only.

38. `subscription\_life\_cycle\_all`

Status change audit trail for subscription_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`subscription_id`	INT	FK to subscription_header_all.id
`previous_status`	INT	Previous subscription status
`new_status`	INT	New subscription status
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of the change
`reason`	VARCHAR(500)	Reason for status change
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
■ INSERT when	Every time the status column in subscription_header_all changes. Record previous_status, new_status, and datetime.	
■ UPDATE when	Never update life cycle rows — append only.	

39. `role\_life\_cycle\_all`

Status change audit trail for role_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`role_id`	INT	FK to role_header_all.id
`previous_status`	INT	Previous role status
`new_status`	INT	New role status
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of the change
`reason`	VARCHAR(500)	Reason for status change
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
■ INSERT when	Every time the status column in role_header_all changes. Record previous_status, new_status, and datetime.	
■ UPDATE when	Never update life cycle rows — append only.	

40. `user\_life\_cycle\_all`

Status change audit trail for user_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`user_id`	INT	FK to user_header_all.id
`previous_status`	INT	Previous user status
`new_status`	INT	New user status
`changed_by`	INT	User ID who performed the change
`changed_on`	DATETIME	Timestamp of the change
`reason`	VARCHAR(500)	Reason for status change
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ INSERT when	Every time the status column in user_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

3. PRODUCTION RULES APPLIED

The following 28 rules from the AI Database Rule Framework were applied during schema generation.

Rule ID	Rule Name	Category	Priority
9	Centralised ID Registry	identity	CRITICAL
2	Two-ID System	identity	CRITICAL
12	Schema-Enforced Workflow Gates	validation	CRITICAL
1	Naming Taxonomy	naming	CRITICAL
7	Indian GST Compliance	taxation	CRITICAL
29	Decimal Money Fields	financial	CRITICAL
44	Three-Tier Settings Cascade	architecture	HIGH
108	Relationship-Scoped Permissions Inline on Linkage Row	architecture	HIGH
77	UNIQUE Constraint on Business IDs	constraints	HIGH
45	Temp-to-Permanent Promotion	architecture	HIGH
3	Three-Layer Data Preservation	audit	HIGH
30	Timestamp Column Naming	naming	HIGH
27	Denormalised Running Balance	financial	HIGH
51	Failed Transactions in Separate Table	financial	HIGH
57	Pre-Computed Analytics at Write Time	financial	HIGH
103	Multi-Tier Transaction Consolidation	financial	HIGH
10	Infrastructure Monitoring via Schema	monitoring	MEDIUM
101	UI Process Registry in Schema	monitoring	MEDIUM
49	Cross-Entity Identity Ledger	architecture	MEDIUM
25	Domain-Specific Tables for Shared Structure	architecture	MEDIUM
37	bigint for High-Volume Transaction PKs	identity	MEDIUM
92	Flat Tables for Fixed Third-Party API Responses	domain	MEDIUM
19	Dynamic Configuration Pattern	configuration	MEDIUM
50	API Failover Logic in Schema	operations	MEDIUM
8	Status Integer Convention	convention	MEDIUM
34	System Singleton Configuration (factory_reset Pattern)	infrastructure	HIGH
96	Hybrid Singleton: Typed + Name-Value	configuration	MEDIUM
107	Framework Tables Coexist Unchanged	architecture	HIGH

4. DEVELOPER NOTES

Business ID Generation	All business IDs (student_id, fee_id, etc.) must be generated from unique_id_header_all. Insert a row to register each entity type with its prefix. Increment last_id on every new record. Never use the auto-increment 'id' column as a business identifier.
Foreign Key References	All foreign keys must reference the integer 'id' PRIMARY KEY column, not the business ID (varchar) column. The business ID is for display only. Join tables using integer id columns for performance.
Archive Table Usage	Copy the full row to the corresponding _archive_all table BEFORE making any UPDATE to a _header_all table. Archive tables store the history of every change. Never DELETE from _header_all — use status instead.
Life Cycle Table Usage	Insert a row into _life_cycle_all every time the status column changes. Store previous_status, new_status, and the datetime of the change. This enables full audit trail of every state transition.
GST Handling	Always store CGST and SGST amounts separately. Never combine them into a single tax column. Calculate at write time and store — never compute at read time. Current rate: CGST 9% + SGST 9% = 18% total (verify with your accountant for current applicable rates).
Closing Balance	The closing_balance column on transaction tables must be updated atomically with the transaction insert. Use a database transaction (BEGIN/COMMIT) to ensure the balance and the transaction record are always consistent. Never update balance separately.
Status Convention	Status 1 = active/success. Status 2 = inactive/failure. Status 3 = deleted/cancelled. Never hard DELETE rows. Set status = 2 or 3 and use soft delete everywhere.
Index Usage	All foreign key columns are indexed. Use the named indexes in your WHERE clauses. Avoid SELECT * in production — always select specific columns to use covering indexes effectively.